

1-2 · 쉘 스크립트 · 실습 1.5H

스크립트 작성 실습

인자·조건·반복·함수·case — 열 단계 실습

수도권ICT 이노베이션스퀘어
데이터 엔지니어링 & RAG 파이프라인 160H · 3기

1.5시간 뒤 손에 남는 것

- ① `greet.sh` — 인자를 받고, 없으면 사용법을 알리고 `exit 1`
- ② `check_file.sh` — 파일 유무로 종료 코드 0 / 1 을 돌려준다
- ③ `scan_logs.sh` — `for` 로 폴더 안 로그를 한 번에 집계
- ④ `report_tool.sh` — 함수·case·while read 에 `set -euo pipefail`

일부러 깨뜨리는 단계가 셋 — 따옴표 빼기 · `set` 옵션 없이 돌리기 · `|| true` 지우기

열 단계 — 스크립트 하나를 키워 가며 문법을 손에 익힌다

각 단계 끝의 「확인 포인트」가 맞으면 다음으로

1~2 · greet.sh	3 · 따옴표 실험	4~5 · 검사·집계	6~9 · 함수·case	10 · ShellCheck
인자 \$1 · 사용법 · exit 1	bash 에서 ls \$f 와 ls "\$f"	check_file.sh · scan_logs.sh	report_tool.sh · while · set	검사기로 마무리

처음 보는 모양 셋 — 치기 전에 알고 간다

모양	뜻	없으면
<code>cat > 파일 << 'EOF'</code>	다음 줄부터 EOF 줄까지 파일에	한 줄씩 echo 로 넣어야
<code>chmod +x 파일</code>	실행해도 된다고 표시	Permission denied
<code>./greet.sh</code>	지금 이 폴더의 greet.sh 실행	command not found

'EOF' 에 따옴표를 씌워야 안쪽 \$1 이 지금 값으로 바뀌지 않고 글자 그대로 파일에 적힌다

greet.sh — 인자를 받고, 없을 때는 막는다

1단계 — 인자를 받는다

./greet.sh 데이터엔지니어 → 안녕하세요, 데이터엔지니어 님!

- name="\$1" — 첫 번째 값을 name 에
- echo "안녕하세요, \$name 님!"
- 인자 없이 실행하면 자리가 빈 채 나온다
- 두 낱말을 주면 첫 번째만 (\$1 이 첫 번째)

VS

2단계 — 없으면 막는다

새벽에 도는 프로그램은 「이름을 안 줬구나」를 판단하지 못한다

- if [\$# -lt 1]; then — 받은 값이 1 보다 적으면
- echo "사용법: \$0 이름"
- exit 1 — 실패 표시를 남기고 멈춘다
- echo \$? → 1 · 홍길동을 주면 0

따옴표를 빼면 이름 하나가 둘로 쪼개진다

1

bash 로 들어간다

zsh 는 쪼개기를 안 해 재현이 안 된다 — 실제로 도는 곳은 bash

2

f="my report.txt"

touch 로 공백이 든 이름의 파일을 만들고 변수에 담는다

3

ls \$f → 오류가 두 줄

my · report.txt 로 나뉘어 각각 찾아본 것 — 파일이 없다는 뜻이 아니다

4

ls "\$f" → my report.txt

따옴표를 씌우면 통째로 하나 — rm "my report.txt" · exit 로 정리

규칙

예외 없이

"\$변수"

새로 나오는 조각 — 표로 먼저 짚고 친다

조각	뜻	단계
if [-f "\$target"]	일반 파일이 있으면 then 쪽으로	4
\${1:-logs}	안 주면 logs 를 쓴다 (기본값)	5
for f in "\$dir"/*.log	찾은 파일을 하나씩 f 에	5
\$(wc -l < "\$f")	줄 수만 값으로 (이름 없이)	5
usage() { ... } · usage	덩어리에 이름 · 부를 땐 이름만	6
local file="\$1"	함수 안에서만 쓰는 변수	6
case "\$mode" in ... esac	갈래는 ;; 로 끝 · *) 는 그 밖	7
while read -r line	done < "\$f" — 한 줄씩	8

grep 은 한 건도 못 찾으면 실패(1)로 끝난다 — count_errors 에 || true 가 붙은 이유

set -euo pipefail — 셸이 각각 무엇을 막는지 겪어 본다

1 set -e

없으면 중간이 실패해도 끝까지 진행 → 있으면 실패한 줄에서 즉시 멈춘다

2 set -u

없으면 오타 변수(user_nmae)가 빈 값으로 조용히 돈다 → 있으면 unbound variable

3 set -o pipefail

없으면 cat 이 실패해도 wc 가 성공이면 종료코드 0 → 있으면 1

4 || true 를 지우면

ERROR 없는 로그에서 grep 이 실패(1) → set -e 가 스크립트를 통째로 멈춘다

셔뱅 바로 아래에 넣는다 — 외우기: -e 넘어가지 마라 · -u 없는 걸 쓰지 마라 · pipefail 중간도 보라

ShellCheck — 조용히 통과하면 검사기가 일하는지 알 수 없다

1 shellcheck *.sh

네 개 모두 아무 말 없이 끝나면 성공 — echo \$? 가 0

2 bad.sh 로 일부러 잡히게

따옴표 없는 변수 · 백틱 · 따옴표 없는 for 그룹을 모아 둔 파일

3 SC2086

Double quote to prevent word splitting — 따옴표 없는 변수

4 SC2006 · SC2231

백틱 대신 \$() · for 그룹에 따옴표

5 등급 넷

error · warning 부터 고친다 — info · style 은 「더 나은 방법」

단계마다 이것이 나오면 다음으로

- ① `./greet.sh` → 사용법: `./greet.sh 이름` · `echo $?` 가 1
- ② `ls $f` 는 두 번 실패, `ls "$f"` 는 `my report.txt`
- ③ `check_file.sh access.log` 뒤 `$?` 는 0, 없는 파일은 1
- ④ `scan_logs.sh` → app1 1줄·1건, app2 2줄·1건
- ⑤ `report_tool.sh summary` · `detail` · `help` 가 모두 동작
- ⑥ `shellcheck *.sh` 가 조용히 끝나고 `echo $?` 가 0

1-2 · 실습 1.5H 마무리

다음은 데이터 처리 자동화 스크립트 (1H)

오늘 만든 조각을 모아 매일 돌릴 수 있는 집계·백업 스크립트로 완성합니다

✓ `count_errors.sh` — 로그 하나의 전체·ERROR 줄 수

✓ `backup.sh` — 날짜가 붙는 백업 폴더

✓ `daily_report.sh` — `set -e`·`set -u` 로 조립

✓ cron 맛보기와 미니 과제

준비할 것

오늘 만든 스크립트와 `logs/` 폴더